



N-CAPAS BY MODULE Y N-CAPAS ALL PROJECT
SISTEMA DE GESTIÓN DE LAVADERO DE CARROS (CAR WASH)

Programa: Ingeniería De Sistemas

Asignatura: Arquitectura De Software

Profesor: Jesús Ariel Gonzales

Estudiante

María Sofia Aljure Herrera

Brayan Smith Bedoya

Corporación Universitaria Del Huila Corhuila

Neiva Huila

Octubre 2025

Introducción

Este taller tiene como propósito construir un sistema CRUD aplicando la arquitectura en N-Capas, trabajando con dos escenarios: All Project y By Module. Para el ejemplo escogimos un Sistema de Gestión de Lavadero de Carros (Car Wash), ya que es un caso sencillo de entender, pero al mismo tiempo permite visualizar cómo un proyecto pequeño puede escalar y crecer en el futuro.

objetivo general

Desarrollar un sistema CRUD aplicando la arquitectura en N-Capas con los escenarios All Project y By Module, usando como ejemplo un Sistema de Gestión de Lavadero de Carros.

objetivos específicos:

- Implementar tres módulos con sus entidades y atributos básicos (Clientes, Vehículos, Servicios).
- Comparar las ventajas y desventajas de All Project vs By Module.
- Mostrar cómo este ejercicio puede escalar a una arquitectura de microservicios.

Arquitectura N-Capas arquitectura en N-Capas es un patrón clásico de diseño de software que organiza el sistema en capas jerárquicas, donde cada capa tiene una responsabilidad específica y conoce únicamente la capa inmediatamente inferior.

Esta arquitectura tiene un flujo típico que es:

1. **Presentación (Controller / API Layer):** esta parte Interactúa con el usuario o cliente externo, recibe las peticiones, valida datos y los redirige a la lógica de negocio.
2. **Aplicación / Servicio (Service Layer):** acá esta la lógica de negocio, define las interfaces (IService) y clases que implementan esa lógica (ServiceImpl).
3. **Dominio / Modelo (Domain Layer / Entity Layer):** en esta parte se definen las entidades o modelos que representan objetos del negocio (ej. User, Product, Order) y se encargan de reflejar reglas básicas del negocio.
4. **Persistencia / Integración (Repository Layer / Data Access Layer):** la persistencia o integración encapsula la lógica de acceso a datos, normalmente implementada con repositorios que extienden de JPA/Hibernate (JpaRepository).

Formas de organizar el código con N-Capas

Cuando aplicamos N-Capas en un monolito o en un modular monolith, existen dos formas principales de organizar los paquetes del proyecto:

N-Capas all project

En este enfoque, todos los controllers, services, repositories y model están agrupados en carpetas globales. Las capas están centralizadas y todos los features comparten esas capas.

Ejemplo de estructura:

- Proyecto único con paquetes por capa:
 - com.app.presentation
 - com.app.application
 - com.app.domain
 - com.app.persistence

Ventajas

- **Simplicidad inicial:** es una estructura conocida, tutoriales y las herramientas la favorecen.
- **Menos artefactos en el build:** el que haya un solo jar/war facilita despliegue inicial.
- **Reutilización centralizada:** las clases utilitarias y modelos se ubican en un solo sitio.

Desventajas

- **Crecimiento en espiral:** a medida que el sistema crece, los paquetes por capa se vuelven enormes y difíciles de navegar.
- **Acoplamiento por tipo:** la tendencia es que módulos funcionales queden dispersos por múltiples paquetes (controlador, servicio, repositorio), dificultando cambios locales.
- **Dificultad para dividir en microservicios:** separar funcionalidades en servicios físicos requiere muchas refactorizaciones y extraer código con riesgo.

Tipos de proyectos indicados

- Aplicaciones pequeñas o medianas donde la cantidad de funcionalidades es reducida.

- Proyectos de uso interno en una empresa.
- Prototipos o MVPs donde lo más importante es salir rápido a producción.
- Aplicaciones académicas o de práctica (proyectos universitarios, laboratorios).

N-Capas by module

En este enfoque, la aplicación se divide en módulos funcionales (features) y cada módulo tiene sus propias capas internas (controller, service, repository, model).

Por ejemplo: module-product tiene product.controller, product.service, product.repository, product.model.

Ejemplo de estructura:

- Módulos (ejemplos): product-module, order-module, user-module
- Dentro de product-module:
 - product.controller
 - product.service
 - product.domain / product.model
 - product.repository
 - product.integration

Como cada módulo tiene sus propias capas internas es responsable por su propio ciclo de vida lógico y puede ser construido, testeado y algunas veces desplegado independientemente si la estructura lo permite.

Ventajas

- **Cohesión alta:** Alta cohesión por módulo.
- **Facilidad para escalar el equipo:** equipos pueden trabajar por módulos con conflictos mínimos.
- **Migración a microservicios más simple:** la delimitación del módulo facilita convertirlo en servicio independiente cuando sea necesario.
- **Mejor entendimiento en contexto del dominio:** navegabilidad intuitiva por feature.

Desventajas

- **Duplicación potencial:** sin una capa común, utilidades o DTOs pueden repetirse si no se gestionan como librerías compartidas.

- **Mayor número de artefactos:** si cada módulo produce su propio jar, el sistema de build y despliegue se vuelve más complejo.
- **Comunicación entre módulos:** si hay dependencias fuertes, se pueden formar acoplamientos cruzados.

Tipos de proyectos indicados

- Aplicaciones medianas o grandes con múltiples funcionalidades y dominios de negocio.
- Proyectos donde trabajan varios equipos en paralelo, cada uno encargado de un módulo.
- Sistemas con una visión de crecimiento a futuro o donde se prevé migrar a una arquitectura de microservicios.
- Aplicaciones que requieren alta mantenibilidad y baja dependencia entre componentes.

Consumo de recursos a nivel de infraestructura

Hay que tener en cuenta que el consumo de recursos a nivel de infraestructura en N-Capas all project y N-Capas by module no es exactamente el mismo, porque la forma en que se organizan afecta la ejecución, despliegue y escalabilidad.

N-Capas all Project

La infraestructura de todo el sistema de all Project corre en un solo artefacto ya sea .jar o .war, también la aplicación es monolítica, lo que significa que se despliega como una unidad en un solo servidor o contenedor y usa una única base de datos compartida para todos los módulos y features.

Así que el consumo de recursos se puede ver de esta manera:

- **Memoria RAM y CPU:** es moderado, ya que solo existe una instancia centralizada.
- **Infraestructura:** solo basta un servidor físico, una máquina virtual o un contenedor Docker.
- **Escalabilidad:** es vertical, si la aplicación crece, se necesita más hardware (RAM, CPU) en la misma máquina.
- **Problema:** aunque solo crezca una parte del sistema (ej: módulo de pagos), se escala todo el monolito, lo cual es costoso e ineficiente.

Como decía anteriormente en los tipos de proyectos indicados es ideal para proyectos pequeños/medianos porque requiere menos infraestructura inicial.

N-Capas by Module

La infraestructura de la aplicación sigue siendo monolítica, pero internamente está organizada por módulos y cada módulo tiene su propia lógica de negocio y capas, lo que facilita desacoplarlos más adelante. Si se da el salto a microservicios, cada módulo podría convertirse en un servicio independiente con su propia base de datos o instancia.

Así que el consumo de recursos se puede ver de esta manera

- **Memoria RAM y CPU:** al inicio, es similar al all project, porque todo se despliega en un único artefacto.
- **Infraestructura inicial:** es igual de sencilla, un servidor o contenedor único.
- **Escalabilidad:** es mucho mejor preparada, puedes escalar por módulo (si se migra a microservicios) y el consumo de recursos puede distribuirse entre varios nodos o contenedores en el futuro.
- **Problema:** requiere más **disciplina en el diseño** y, al crecer, puede implicar **mayor consumo global** si cada módulo se despliega separado (ya que cada microservicio necesita memoria, CPU, base de datos o al menos un esquema independiente).

Como decía anteriormente en los tipos de proyectos indicados es ideal para proyectos medianos/grandes porque optimiza la escalabilidad futura, aunque a nivel de infraestructura el costo puede subir si se migra a microservicios.

Desarrollo de crud con escenario 1 y escenario 2

En este primer taller vamos a construir un sistema CRUD basado en la arquitectura de N-Capas, aplicando los escenarios By Module y All Project. El ejemplo escogido es un Sistema de Gestión de Lavadero de Carros (Car Wash), ya que es práctico y fácil de entender, pero al mismo tiempo permite proyectar cómo un negocio sencillo en el futuro, pueda desplegarse, optimizarse y escalarse de forma separada como microservicio.

Módulos y Entidades

Para el desarrollo de la crud usaremos 3 módulos y sus entidades

1. **Clientes:** para registrar y gestionar la información de los que utilizan el servicio.

Entidad: cliente

- IdCliente

- NombreCompleto
- Documento
- Telefono
- Correo

2. **Vehículos:** para identificar y controlar los autos que ingresan al lavadero.

Entidad: Vehículo

- IdVehiculo
- Placa
- TipoVehiculo (carro, moto, camioneta, etc.)
- IdCliente

3. **Servicios:** para administrar los tipos de lavado.

Entidad: servicio

- IdServicio
- NombreServicio
- Descripcion
- Precio
- DuracionEstimada

Aunque empezaremos solo con estos tres módulos, primero, implementaremos el escenario All Project, en donde todos los módulos y sus atributos estarán organizados dentro de una misma estructura de capas, pero el sistema está pensado para ser escalable. Esto significa que, en el futuro, se podrán agregar más módulos y entidades como empleados, pagos, reservas etc, sin necesidad de rehacer todo el proyecto. Así que en ese orden de ideas la arquitectura perfecta seria N capas by module ya que nos permitirá mantener una estructura organizada, flexible y preparada para crecer conforme aumenten las necesidades.

¿Qué hace este sistema all project?

Tu sistema es un **CRUD en Go (All Project)** con tres módulos principales:

- **Clientes**
- **Vehículos**
- **Servicios**

Cada módulo tiene:

- **Entity** → Define la estructura de datos (ej: Cliente con nombre, documento, correo).
- **Repository** → Se conecta con la base de datos (consultas con GORM a MySQL).

- **Service** → Contiene la lógica de negocio (ej: antes de guardar un cliente validar que no exista otro con el mismo documento).
- **Handler** → Expone endpoints HTTP para que el frontend o Postman hagan peticiones REST (ej: GET /clientes, POST /vehiculos).

Flujo

1. El **usuario** (tú, Postman, o Angular) hace una petición HTTP (ej: POST /clientes con JSON de un cliente).
 2. Esa petición llega al **Handler** correspondiente (ej: ClienteHandler).
 3. El Handler llama al **Service**.
 4. El Service a su vez llama al **Repository**.
 5. El Repository usa **GORM** para ejecutar SQL sobre la DB en MySQL (XAMPP).
 6. La respuesta fluye en sentido inverso hasta que el **Handler** devuelve JSON al cliente.
-

Diferencia All Project vs By Module

All Project (monolítico)

- Todo está en **una sola app Go**.
- Solo tienes **un main.go** que arranca todo.
- Una sola conexión a la DB (db/connection.go).
- Todos los módulos viven dentro de la misma instancia del servidor (localhost:8080).
- Ejemplo:
 - GET http://localhost:8080/clientes
 - GET http://localhost:8080/vehiculos
 - GET http://localhost:8080/servicios

By Module (modular / simulando microservicios)

- Cada módulo sería **su propio proyecto** (clientes/, vehiculos/, servicios/).
- Cada módulo tendría su propio **main.go** y podría correr en un **puerto diferente**.
- Podrían compartir la misma DB en XAMPP, o tener DB separadas (depende de diseño futuro).

La **diferencia principal**: en All Project corres una sola app; en By Module corres varias apps (cada módulo puede vivir independiente, ideal para microservicios).

Diferencia principal entre *All Project* y *By Module*

All Project: todo el código está en carpetas globales (`entity`, `repository`, `service`, `handler`) y el `main.go` usa todo junto.

By Module: cada módulo (ej. `vehiculos`, `clientes`, `servicios`) **tiene sus propias capas internas** (`dto`, `entity`, `repository`, `service`, `handler`). Esto hace el sistema más **modular y mantenible**, porque cada módulo es casi independiente.